

Architectural Specification for the Dungeon Agents Multi-Agent Simulation: A Deep Research Report

Foundations of the Simulation Engine: The Environment and State Machine

The core of the Dungeon Agents simulation is an environment that must maintain absolute ground truth while selectively exposing partial information to heterogeneous agents. This requirement necessitates a strict separation between the physical world state and the perceptual layers of each agent. In a production-grade implementation, the environment is best modeled as an immutable event store, where every movement, item interaction, and communication is recorded as a discrete event before being projected into the current board state.¹

World State and Grid Management

The 8×8 grid serves as the spatial foundation of the simulation. For a project focused on traces rather than design complexity, the grid must be a formal state machine. Each cell on the board maintains its own state: empty, wall, item (key), or gate (locked door). The logic for unlocking the gate must be centralized within an Environment Orchestrator to ensure that the "key-in-possession" check is authoritative and not susceptible to agent hallucinations.

Component	Responsibility	Technical Implementation
Grid Logic	Spatial constraints and cell properties	Coordinate-based state matrix (NumPy/TypeScript Array)
Perception Engine	Fog of war calculation (3×3 adjacent)	Viewport masking based on agent coordinates
Interaction Layer	Validating move, pick-up, and	Deterministic validator nodes in

	unlock actions	LangGraph
Dungeon Master	Stale state reporting and global oversight	Delayed view projection from the event store

The introduction of the Dungeon Master (DM) agent adds a layer of complexity designed to test agent communication. The DM operates on a stale state of N turns, necessitating an architecture where the global event log is indexed by turn number, allowing the DM to query the state as it existed in the past.³ This creates a situation where agents must reconcile their immediate, high-fidelity local observations with the DM's global but outdated information, a common pattern in distributed systems where local nodes often have more recent data than a central controller.⁴

The Asynchronous Game Loop

The game loop must handle the turn-based logic with a specific focus on message latency. If Agent A sends a message at Turn 10, Agent B should not receive it until Turn 11.³ This requires a message queue that is integrated into the simulation's temporal logic. A central orchestrator manages the tick of the simulation clock, ensuring that "intentions" are gathered from all active agents before state transitions are applied. This prevents race conditions and ensures that the traces reflect a consistent ordering of events.

Observability Stack: Traces, Instrumentation, and Epistemic Metrics

The core requirement of the project is not the success of the agents, but the quality of the traces and the legibility of the diagnosis. A standard log file is insufficient for identifying "reasonable decisions based on stale information".³

OpenTelemetry and the Langfuse Integration

The industry standard for capturing complex agentic behavior is OpenTelemetry (OTel). OTel provides a structured way to emit spans for every LLM call, tool execution, and state transition.¹¹ Langfuse serves as the ideal backend for these traces, as it natively supports OTel and provides specialized visualizations for agent graphs.¹³

Every agent turn should generate a hierarchical trace structure:

1. **Parent Span:** The complete turn cycle.

- 2. **Perception Span:** The process of the agent receiving its 3×3 grid observation.
- 3. **Reasoning Span:** The internal CoT (Chain of Thought) or planning phase.
- 4. **Action Span:** The tool call and its immediate result from the environment.
- 5. **Metadata Attributes:** Latency, token usage, and most importantly, the "Belief State".¹³

Attribute Type	Key Examples	Diagnostic Purpose
Trace-Level	session_id, run_variant, agent_type	Grouping runs for comparison and regression testing
Observation-Level	model_name, temperature, tool_name	Identifying model-specific failure patterns
Metadata-Level	belief_coordinates, known_items, perceived_fog	Tracking the divergence between belief and reality

To ensure filterable metadata in Langfuse, attributes must be prefixed according to OTel conventions (e.g., langfuse.trace.metadata.belief_state).¹³ This enables a human diagnoser to query for all runs where an agent’s belief about the exit’s location was more than N units away from the ground truth.

Event Sourcing for Multi-Agent Simulations (ESAA)

The Event Sourcing for Autonomous Agents (ESAA) architecture provides a rigorous framework for auditability. In this model, the agent does not have direct write access to the simulation state.¹ Instead, it emits "Intentions" (e.g., "I intend to move North"). These intentions are validated by an Orchestrator and appended to an immutable Event Store (activity.jsonl).¹ The current board state is merely a projection of this event log.

This architecture enables "Deterministic Replay," where a failed run can be re-executed from any specific point in the timeline to test "what-if" scenarios.¹ If an agent makes a fatal error at Turn 30, the diagnoser can modify the agent's prompt or tools at Turn 29 and see if the outcome changes.

Measuring Epistemic Divergence and Theory of Mind

In production multi-agent systems, the most subtle bugs occur when information fails to propagate correctly between agents. This is termed "Interaction Uncertainty" or "Epistemic Divergence".⁴

Quantifying the Belief-Reality Gap

To help a human understand "Why did it happen?", the instrumentation must track the agent's internal world model. If Agent A believes a door is locked, but Agent B has already unlocked it, we have an epistemic gap. We can measure this gap using a Divergence Metric:

$$D_{epistemic} = \sum_{c \in Grid} |Belief_{Agent}(c) - Reality(c)|$$

By logging this value at each turn, the dashboard can highlight "Spikes of Divergence" where an agent's mental model significantly drifted from reality.¹⁶ This allows a human to quickly identify the turn where a communication failure or a missed observation occurred.

The Legibility Layer: Designing the Diagnosis Tool

The final component of the project is a "Legibility Layer" that transforms raw JSON traces into human-interpretable insights. A professional diagnosis tool must go beyond a simple list of logs to provide a temporal and causal analysis of the run.²⁰

Visualization Patterns for Multi-Agent Traces

Based on Agent-Based Modeling (ABM) visualization guidelines, the interface should prioritize spatial and temporal clarity.²²

1. **Dual-Perspective Replay:** A split-screen view showing the Ground Truth board on the left and the Agent's Subjective Map on the right. This immediately surfaces "Fog of War" discrepancies.²¹
2. **Causal Graph View:** Using libraries like React Flow or D3-Force, the tool should reconstruct the causal dependencies between actions.²⁴ If an agent fails to reach the exit, the causal graph can trace backward from the failure to find the "root cause" node—for example, a tool failure that went unnoticed or a message that was dropped.²⁴
3. **Gantt-Style Timeline:** A visualization of concurrent agent activities. This reveals bottlenecks where one agent was idle waiting for a message from another.²³
4. **Confidence and Belief Heatmaps:** Mapping the agent's probabilistic belief state onto the grid. A "Heat Map Vector Viewer" can show how different agents perceive the importance of different cells over time.¹⁶

Designing for Failure Mimicry

The "Dungeon Agents" prompt encourages the design of tool calls that mimic real-world failure

modes. This requires a dashboard that can categorize these failures:

Failure Category	Mechanism	Instrumentation Signal
Stale State	Agent acts on data from $T - N$	High divergence_score for visited cells
Communication Drop	Message sent but not received/processed	Mismatch between intent_out and belief_in
Tool Misinterpretation	Model ignores tool error output	Action node continues despite error metadata
Goal Drift	Agent forgets objective due to context loss	Shift in latent_goal attribute in reasoning trace

By categorizing failures in this way, the diagnosis tool can automatically generate a "Causal Incident Report" for every failed run, answering the project's core questions: What happened? Why? And what should change?.³

The "High-Signal" Recommended Stack

This stack prioritizes the advanced reasoning of the research report while staying within the time limit by sticking to **one language** (Python).

- **Language: Python 3.12+.** It is the fastest for agent logic and has the most robust integration for the recommended tools.
- **Agent Framework: LangGraph.** This is the most important recommendation from the research. It natively handles the cycles (looping to find a key) and state persistence required for a dungeon, which standard LangChain struggles with.
- **Observability: Langfuse.** The prompt explicitly asks for an observability tool , and Langfuse provides the best "out of the box" visualization for agentic graphs.
- **State & Validation: Pydantic V2.** Use Pydantic models for your "Event Schema". This ensures your traces are structured and clean—a key evaluation

point.

- **Legibility Layer (UI): Streamlit** (instead of Next.js). While the research suggests Next.js, building a React dashboard in 60 minutes is risky. Streamlit allows you to build a functional, opinionated diagnostic tool in Python in 15 minutes

Hard Rules to keep in mind while building:

- **Sequential Game Loop:** Do not make the agents act at the same time; gather intentions and apply state transitions turn-by-turn.
- **Communication Lag:** Ensure messages sent at Turn N are only visible at Turn N+1.
- **Immutable Logs:** Even if you don't use a database, save every step to a `runs.jsonl` file to fulfill the "Event Sourcing" recommendation for deterministic replay.